DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment 5

Student Name: Om Prakash reddy **UID: 23BAI70296**
Branch: AIT_CSE **Section/Group: 23AIT_KRG_G2**
Semester: 6th **Date of Performance: 19-02-2026**
Subject Name: Full Stack II **Subject Code: 23CSH-382**

1. **Aim:**
   To verify the correctness and reliability of the EcoTrack React application by writing automated tests using Jest and React Testing Library, and by analyzing application behavior using debugging tools.
2. **Objective:**

After completing this experiment, the student will be able to:

1. Understand the purpose of automated testing in frontend applications
2. Write unit tests for JavaScript utility functions using Jest
3. Use different Jest matchers to validate expected outputs and behaviors
4. Test React components using React Testing Library
5. Verify UI rendering by querying elements from the DOM
6. Implement asynchronous testing using findBy and waitFor methods
7. Apply mocking to simulate API or external data responses in tests
8. Perform snapshot testing to detect unintended UI changes
9. Debug failing tests and application logic using browser Developer Tools and breakpoints
10. Analyze application behavior and errors systematically rather than manual checking

3. **Implementation/Code:**

ProductApi.js:

```
export const fetchProducts = () => {
  return Promise.resolve({
    id: 1,
```

```
    name: "Default",
    price: 1000
  });
};
```

Math.js:

```
export const add = (a, b) => a + b;
```

Math.test.js:

```
import { add } from "./math";

test("adds two numbers", () => {
  expect(add(2, 3)).toBe(5);
});
```

Greetings.js:

```
const Greeting=({name})=>{
    return (
        <div>
            <h1>Hello {name}</h1>
        </div>
    );
};
export default Greeting;
```

Greetings.test.js:

```
import { render, screen } from "@testing-library/react";
import Greeting from "./Greetings";

test("renders greeting message", () => {
  render(<Greeting name="John" />);
  expect(screen.getByText("Hello John")).toBeInTheDocument();
});
```

Product.js:

```
import React, { useState, useEffect } from "react";
import { fetchProducts } from "../api/productApi";

const Product = () => {
  const [product, setProduct] = useState(null);

  useEffect(() => {
```

```
    fetchProducts().then(setProduct);
  }, []);

  if (!product) {
    return <p>Loading...</p>;
  }

  return (
    <div>
      <h2>{product.name}</h2>
      <p>Price: {product.price}</p>
    </div>
  );
};

export default Product;
```

Product.test.js:

```
import { screen, render } from "@testing-library/react";
import Product from "./Product";
import * as api from "../api/productApi";

jest.mock("../api/productApi");

test("renders product", async () => {
  api.fetchProducts.mockResolvedValue({
  id: 1,
    name: "phone",
    price: 2000
  });

  render(<Product />);

  const productName = await screen.findByText("phone");

  expect(productName).toBeInTheDocument();
});
```

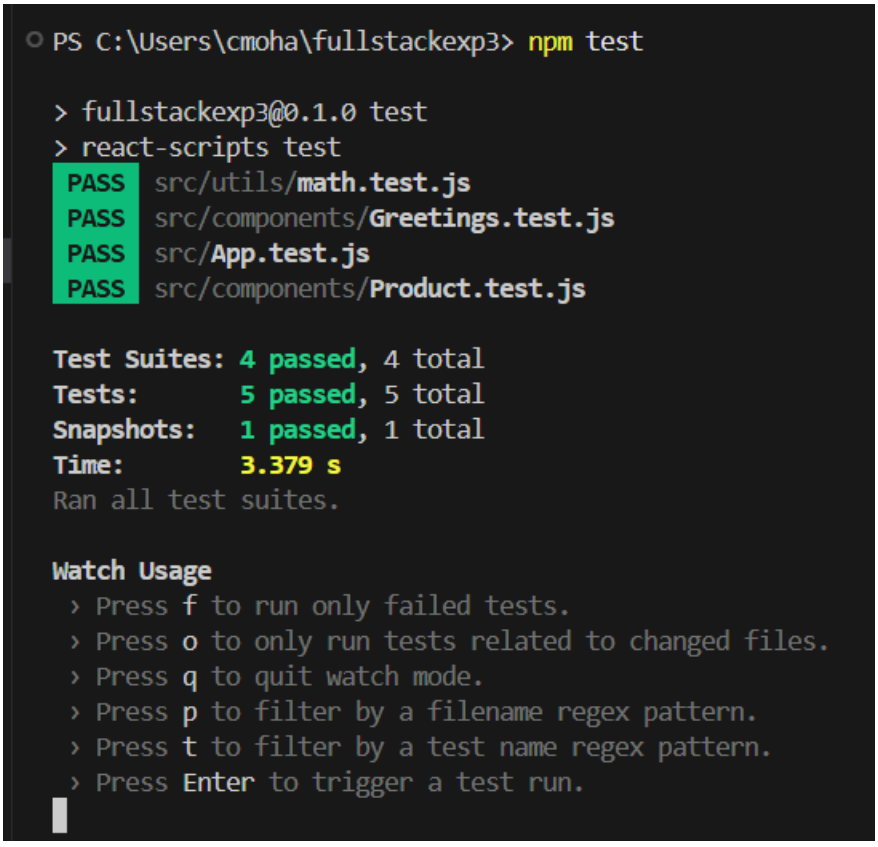
Header.jsx:

```
const Header = () => {
  return (
    <header>
```

```
      <h1
        style={{
          color: "#ffffff",
          backgroundColor: "#001780",
          padding: "20px",
          margin: 0,
          textAlign: "center",
          letterSpacing: "1px"
        }}
      >
        Welcome to Daily Logs
      </h1>
    </header>
  );
};

export default Header;
```

4. Output:



5. Learning Outcome:

1. Understand the importance of automated testing in modern frontend development to ensure application reliability and maintainability.
2. Write unit tests using Jest to validate JavaScript utility functions and core logic independently.
3. Apply different Jest matchers such as toBe, toEqual, toContain, and toBeTruthy to verify expected outputs and behaviors.